CONFIDENTIAL · INTERNAL ENGINEERING BRIEF

Larry

MVP Engineering Roadmap

A brutally honest gap analysis between the product vision and the current codebase, with prioritised responsibilities assigned to each engineer. MVP target: Saturday 28 March 2026.

Prepared by
Fergus · Anton · Joel

Date
March 2026

Target
7 Days to MVP

Version
MVP Sprint

SECTION 01

Where We Are vs. Where We Need to Be

Every feature from the product spec mapped against the current codebase. No spin.

The AI pipeline is calling the wrong OpenAI endpoint
The model name was recently corrected to `gpt-5.4-mini`, but the code is calling `/v1/chat/completions`. OpenAI's newer flagship models (GPT-5 family) require `/v1/responses` instead. Every AI call will return a 404 error in production. Fix: change `chat_completions_endpoint` with a proper message array. Until this is fixed, every transcript, flow message, and chat command fails. Back is a keyword matching stub.

FEATURE LABEL	PRODUCT VISION	CURRENT STATE	STATUS
AI router	LLM extracts tasks, deadlines, owners, blockers from any signal	Model name fixed to `gpt-5.4-mini`, but the API call targets `/v1/chat/completions` instead of `/v1/responses`. Standard API rejects GPT-5 models. Fix: use the Responses endpoint, causing a network error on every call	Broken
Signal ingestion: Slack	Continuously ingest Slack messages, normalize into structured events	OAuth + webhook handler fully built. Signature verification in place	Done
Signal ingestion: Calendar	Calendar events trigger prep/follow-up updates	OAuth + webhook built. Watch registration exists but expires after 7 days with no renewal	Partial
Signal ingestion: Email	Larry reads emails to update task state, drafts and monitors responses	Email connector is a stub, the Inbox URL returns a fake OAuth URL. No real provider. Email response monitoring loop does not exist	Missing
Event normalization engine	Raw signal → structured canonical event with confidence score	Pipeline exists and works. Source, actor, confidence, timestamp all captured.	Done
Action extraction engine	Structured event → new/updated task objects with reasoning	Worker pipeline built but LLM call is broken (see above). Policy engine and intervention logic complete.	Broken
State & dependency engine	Update task graph, sync dependencies, recalculate downstream impact	DB schema and basic task dependencies exist. No downstream recalculation on task completion. No auto-notify need owner.	Partial
Execution health engine	Reliability score based on owner behavior, latency, slippage, silence	Basic linear arithmetic rule (score deadline − progress × inactivity). No owner behavioral tracking or slippage patterns.	Partial
Autonomous intervention	Reminders, nudges, escalations sent automatically	Escalation scan in worker runs on settimeout (not production-grade). Escalates to assignee but no manager hierarchy.	Partial
Human validation gate	Action Centre: approve/reject deadline, scope, ownership changes	Fully built. Approve / reject / override with correction feedback works end-to-end.	Partial
Correction & calibration	Human corrections feed back to adjust confidence thresholds per tenant	Correction feedback stored. Threshold nudge (±0.02) implemented. Not yet visibly tuning outcomes.	Partial
Project creation: Manual	User sets up tasks, owners, deadlines manually	Works. Projects and tasks can be created via API and UI.	Done
Project creation: Chat	User describes project to Larry, Larry proposes full structure for approval	Larry chat exists but sends a freeform command. It does not propose a structured project for approval. No "create project from conversation" flow.	Missing
Project creation: Meeting	Upload/record a meeting → Larry builds the project structure from it	Transcript parse exists and extracts actions. But it does not build a full project structure. It only creates proposed actions. No meeting recording.	Partial
Voice input	Chat with Larry by voice; dictate project setup	Not implemented.	Missing
Gantt chart	Expandable task hierarchy, cascades-aware drag, with dependency arrows	Basic Gantt exists. No expand/collapse subtree hierarchy. Clicking bars not wired to task detail.	Partial
Dashboard: demand vs. resolved chart	Trend widget showing whether the team is keeping up with incoming work	Done widgets. Stacked charts exist. Report snapshot API exists but no PDF/PPT export.	Partial
Report generation (PDF / PPT)	"Generate report" button exports PDF or PowerPoint	Not implemented. Button not present.	Missing
Meetings: transcription	Records and transcribes meetings automatically	Text paste only. No audio recording or Whisper transcription.	Missing
Meetings: action extraction	Larry extracts tasks, deadlines, owners from meeting	Pipeline built but LLM is broken. Meeting notes stored correctly.	Broken
Larry chat persistence	Past conversations accessible from Chats sidebar	DB tables exist. The UI never reads or writes to them, so all history is in memory and lost on refresh.	Missing
Real-time UI updates	Action Center and board update live as worker processes jobs	No WebSocket or polling. User must manually navigate away and back to see worker results.	Missing
Dependency auto-notification	When task completes, notify owner of the next dependent task	DB schema supports it. No trigger implemented.	Missing
Manager escalation	Escalate to assignee AND their manager on deadline breach	Escalation scan sends to assignee only. No manager role or notification hierarchy.	Partial
Notification center	Bell icon with readable, dismissable alerts	Notification bell exists, read_at column exists. Basic list rendered. Not dismissable in UI.	Partial
Documents page	Meeting summaries and generated reports in sidebar	Placeholder only. Empty state with link to Meetings.	Missing

SECTION 02 · ENGINEER 1

Fergus

Platform Engineer

Bridge work: features that span the entire stack from DB to UI

3 PR TASKS · 5 PT TASKS · 7 Days WORKDAYS

Fergus's mandate
Fergus owns the product's nervous system, the parts that cross all layers. He unblocks both Anton and Joel by fixing the AI pipeline first, then delivers the cross-cutting product features. Chief toward project creation and real-time updates, and keeps the infrastructure from falling apart in prod.

DAYS 1–2: UNBLOCK EVERYTHING (22–23 MAR)

1	**Fix the OpenAI API endpoint** · P0 `/v1/chat/completions` → `/v1/responses`. And the code is posting to `/v1/responses`. OpenAI's Responses API with `gpt-5.4-mini`. Standard API requests do not have access to this model. Fix: change `chat_completions_endpoint` to `/v1/responses`, update `extract_from_transcript` and `generate_response` to read `output[0].content[0].text` instead of `choices[0].message.content`. Verify by pasting a transcript and checking the worker logs for successful JSON parsing. Back is a keyword stub until this ships. File: `app/services/openai_client.py`, `app/services/llm_router.py`	Broken now	22–23 Mar
2	**Worker environment parity** · P0 The worker is not guaranteed to have cloud-flavored vars the API sees local Docker. These are talking to different databases. Fix: wonder to use local DB in dev. Add a startup check that logs `DATABASE_URL` prefix and exits if it looks wrong. File: `app/core/worker.py`	Broken now	22–23 Mar
3	**Real-time Action Center polling** · P0 After user processes a commitment, the card has no idea if anything happened. Add a polling loop on the Frontend that queries `/api/v1/signals/extracted?processed=true` every 4 seconds for 30 seconds after any action. Stop polling on the first result or timeout. This is the single biggest demo killer right now. File: `app/actions/page.tsx`, `app/actions/components/ActionCard.tsx`	Missing	22–23 Mar
4	**Larry "create project from conversation" API** · P1 Add a new route `/api/v1/projects/from-conversation`. Which finds the LLM extracts a project name, tasks, owners, deadlines, and dependencies from a chat transcript, creates a structured project proposal (draft), tasks only, target, stores it as a pending agent action, without auto-creating. Route to human approval in the Action Centre. File: `app/services/llm_router.py`, `app/services/action_centre.py`	Missing	24–25 Mar
5	**Project proposal approval → project creation** · P1 When a project creation action is approved in the Action Centre, execute it: create the project, create all tasks, create dependencies, set deadlines, set dependencies. Write the worker's side execution path to handle `project_plan` in `execution_payload`. File: `app/actions/handlers.py`, `app/services/action_centre.py`	Missing	24–25 Mar

DAYS 3–4: INFRASTRUCTURE RELIABILITY (26–27 MAR)

6	**Move escalation scan to BullMQ repeatable job** · P2 The escalation scan currently runs on `settimeout` inside the worker process. If the worker restarts, the scan stops. Move it to a BullMQ repeatable job (`repeat: { pattern: "*/5 * * * *" }`). This is production-hygiene that will embarrass you in a live demo if it fires mid-restart. File: `app/core/worker.py`, `app/jobs/escalation.py`	Fragile	26–27 Mar
7	**Dead letter queue for failed jobs** · P2 Failed BullMQ jobs vanish by default after retries. Add a DLQ so jobs fail after 3 retries, write to a `failed_jobs` DB table with error, payload, and timestamp. Surface count in the health endpoint so silent failures become visible. File: `app/core/worker.py`, `app/jobs/queue.py`	Missing	26–27 Mar
8	**Calendar webhook renewal job** · P2 Google Calendar watch channels expire after 7 days. There is currently zero renewal logic. Add a BullMQ repeatable job (every 5 days) that re-registers the watch for all active tenants. Without this, calendar signals silently stop arriving after one week. File: `app/services/calendar_ingestion.py`, `app/jobs/queue.py`	Missing	26–27 Mar

SECTION 04 · ENGINEER 2

Anton

Backend Engineer

The capabilities layer, covering everything the customer touches and judges the product by

2 PR TASKS · 6 PT TASKS · 7 Days WORKDAYS

Anton's mandate
The product vision is rich and visual. The current UI is functional but thin, missing the Gantt drill-down, report export, persistent chat, and real-time updates. Anton finishes first: takes Larry's broken tasks, builds Larry's to close that gap and make the interface feel as smart as the engine underneath.

DAYS 1–3: CORE UI GAPS THAT MAKE EVERY DEMO LOOK BROKEN (22–24 MAR)

1	**Persistent Larry chat history** · P0 The `/v1/conversations` and `/v1/conversations/{id}/messages` tables exist in the DB and are populated, but the UI never reads or writes to them. Every page refresh wipes the conversation. Wire `GET /v1/conversations` to load the sidebar on mount, `GET /v1/conversations/{id}/messages` on thread open, `POST /v1/conversations` on first message. The Chats page should then display the list of past conversations. Files: `app/chats/page.tsx`, `app/components/LarryChat.tsx`, `app/services/larry.ts`, `app/backend/routers/conversations.py`	Missing	22–23 Mar
2	**Action Centre: show signal source context** · P0 Cards show "Slack message" or "Calendar event" but not what it came from. A PM needs to see which Slack message or meeting triggered this, the user text, and the agent's rationale. Add a collapsible "Source" section on each action card that pulls from `signal_id` and `extracted_action.reasoning`. Backend: `GET /v1/actions/{id}/context`. Files: `app/actions/components/ActionCard.tsx`, `app/backend/routers/actions.py`	Missing	22–23 Mar

DAYS 4–5: PROJECT DRILLDOWN & EXPORT PDF/DOC

3	**Gantt: expand/collapse subtree hierarchy** · P1 The product spec explicitly calls for expandable task hierarchy with cascades-aware drag and drop, sub-timelines. Current Gantt is flat. Add collapsed/expanded per parent task, dependency lines between bars, and a "Generate Report" button that calls `GET /v1/projects/{id}/report` and displays slippage, progress, priority, commentary. This is one of the most visible screens in demos. File: `app/components/GanttChart.tsx`, `app/projects/[id]/page.tsx`	Partial	24–25 Mar
4	**Dashboard: stacked charts per workstream and per assignee** · P1 The product sheet specifies two stacked bar charts on the dashboard: one grouped by workstream, one grouped by assignee, both showing "Demand" vs "Resolved" over time. Backend already returns `workstreams` and `by_assignee` in `GET /v1/dashboard/summary`, but the frontend only `console.log`s the response. Add `recharts` stacked bar charts using those existing fields. File: `app/dashboard/page.tsx`, `app/components/DashboardCharts.tsx`	Partial	24–25 Mar
5	**Notification centre: dismiss & mark read** · P1 Bell icon exists, but notifications cannot be read or dismissed. The `read_at` column exists in the DB and there is a `PATCH /v1/notifications/{id}/read` endpoint already built. Wire up click-dismiss / mark read + remove from unread count. Add a "Mark all read" button. The unread count should update in real-time after each action. File: `app/components/NotificationBell.tsx`, `app/hooks/useNotifications.ts`	Partial	24–25 Mar

DAYS 6–7: REPORT EXPORT & DOCUMENTS

6	**Generate Report button: PDF export** · P1 The product spec explicitly calls for "Generate Report" button to export PDF or PPT. Add a "Generate Report" button to the dashboard that calls the backend PDF generation endpoint (currently with zero UI), receives the PDF blob in response, triggers a browser download. This is a demo, impressive ship moment. File: `app/dashboard/page.tsx`, `app/services/pdf_generator.ts`	Missing	26–27 Mar
7	**Documents page: surface meeting summaries & reports** · P2 Currently a placeholder. The `meetings` table has summaries, action counts, and dates, and the backend has `GET /v1/meetings` and `GET /v1/meetings/{id}` endpoints. Add a list with dates, counts, and an "Open" button. This makes the Documents page live and ties to the meetings engine. File: `app/documents/page.tsx`, `app/components/MeetingCard.tsx`	Missing	26–27 Mar
8	**Chat-based project creation UI** · P2 Backend `POST /v1/projects/from-conversation` is done (builds the frontend). "New Project" flow with a "Set up with Larry" tab. This task ships a text-to-project. Larry returns a structured proposal (name, tasks, owners, deadlines) for review before the user approves and creates. This is the flagship onboarding moment. File: `app/projects/page.tsx`, `app/components/CreateProjectModal.tsx`	Missing	27–28 Mar

SECTION 04 · ENGINEER 3

Joel

Backend Engineer

The engine room. AI pipeline, integrations, data integrity, and execution intelligence

2 PR TASKS · 6 PT TASKS · 7 Days WORKDAYS

Joel's mandate
Joel owns what makes Larry actually intelligent: the AI pipeline quality, the email integration (the only connector that is currently a complete mock), the dependency notification system, and the reliability of the execution engine. The algorithm spec is detailed and Joel's job is to implement it faithfully.

DAYS 1–2: MAKE THE AI ACTUALLY WORK (22–23 MAR)

1	**Rewrite the LLM extraction prompt** · P0 Current prompt treats the transcript as one blob and expects tasks to be back strangers. The product vision requires the full project context: existing tasks, owners, deadlines, recent activity. Rewrite `extract_from_transcript` to accept `project_context` and `today` and return a strict JSON array with fields: `title` (to be explicitly extract task title, owner name, deadline, dependency mentions, confidence (0–1), and `ambiguity_flag` (true if multiple interpretations exist). Test against 3 real meeting transcripts before merging. File: `app/services/llm_router.py`, `app/services/prompts.py`	Broken	22–23 Mar
2	**Wire transcript extraction → actual task creation** · P0 Per the algorithm spec, after extraction and approval, tasks should be created in the system. Currently the worker stores extracted actions but does not create tasks. Add a handler that interpreted the executed part of the worker: when an action of type `create_tasks` is approved, insert tasks in the task table. When `project_update` is approved, update the task. This closes the loop from meeting to actionable work. File: `app/actions/handlers.py`, `app/services/action_centre.py`	Missing	22–23 Mar

DAYS 3–5: EMAIL CONNECTOR (REAL IMPLEMENTATION)

3	**Real email OAuth connector (Gmail)** · P1 The email connector currently returns a fake OAuth URL and stores mock credentials. Implement real Gmail OAuth2 using the Google OAuth flow (same pattern already working for Calendar). On callback, exchange code for access + refresh tokens, encrypt and store in `integration_accounts`. Add cron job to refresh expiring tokens. Gmail API: `users.messages.list` with `q=newer_than:1d` for polling, `users.messages.get` for individual messages. Start with polling, not push (push requires public HTTPS endpoint). File: `app/services/email_ingestion.py`	Missing	24–25 Mar
4	**Email response monitoring loop** · P1 Outbound emails sent by Larry are fire-and-forget. Add a cron job (`*/30 * * * *`) that checks `integration_events` for `email.sent` events with no `email.response_received` within 24 hours, matches by thread ID, and emits `email.response_received`. Feed this into the commitment ledger: mark commitment as fulfilled if recipient responded, escalate if overdue. Without this, Larry cannot know if a request was ever answered. File: `app/services/email_ingestion.py`, `app/jobs/queue.py`	Missing	24–25 Mar

DAYS 5–7: DEPENDENCY ENGINE & EXECUTION HEALTH

5	**Dependency completion notification** · P1 When a task is marked complete and has dependent tasks, automatically notify the owner of the next task in the chain. This is explicitly specified in the product list and in the algorithm (stage 6: "Notify next owner"). Add a trigger in `PATCH /v1/tasks/{id}` when `status = 'done'`. Use `dependency_type = 'blocks'` and create notifications for downstream owners. File: `app/services/dependency.py`	Missing	26–27 Mar
6	**Improve risk scoring with owner behavior signals** · P2 Current risk scoring is a simple linear rule (score deadline − progress × inactivity). Add owner behavioral signals: latency patterns, silence duration, and slippage history. The LLM hallucinated numbers here; these signals are deterministic and more reliable. Pull from `integration_events` to compute average response time, silence days, and slippage count (past tasks), and silence penalty if no comment in 5 days. Store signal breakdown in `risk_signals_fired` for explainability. File: `app/services/execution.py`, `app/services/dependency.py`	Partial	26–27 Mar
7	**Manager escalation hierarchy** · P2 The product spec says escalations go to both the assignee AND their manager. There is currently no manager relationship in the data model. Add an optional `manager_user_id` FK to `users`, update escalation logic to send notifications to both. This is a 2-line schema change and a few lines of logic. File: `app/services/escalation.py`, `app/services/notifications.py`	Partial	26–27 Mar
8	**PDF report generation endpoint** · P2 The product spec calls for "Generate report as PDF or PPT". Build a `GET /v1/projects/{id}/report?format=pdf` endpoint. Aggregate: project name, tasks by status, slippage, priority, generate a structured report. Jinja2 + WeasyPrint, clean layout, clear, risk summary, overdue tasks, and this week's activity. Anton wires the download button on the frontend. PPT can come in a follow-up sprint. File: `app/services/reports.py`, `app/jobs/queue.py`	Missing	26–27 Mar

SECTION 05

7-Day MVP Sprint

MVP Target: Saturday 28 March 2026. PR tasks must ship by Monday 23 March. Every day counts.

ENGINEER	DAYS 1–2 FOUNDATION (22–23 MAR)	DAYS 3–4 CORE FEATURES (24–25 MAR)	DAYS 5–6 POLISH & INTEGRATIONS (26–27 MAR)	DAY 7 MVP READY (28 MAR)
Fergus	· Fix OpenAI API endpoint (P0) · Fix worker DB vs PG2 (P0) · Real-time Action Centre polling (P0)	· Larry "create project from conversation" API (P1) · Project proposal → project creation (P1)	· Move escalation to BullMQ repeatable · Email after queue · Calendar watchlist renewal job	· E2E smoke test all 3 engineer paths · Load test core endpoints · Sentry environment hardening
Anton	· Persistent Larry chat history (P0) · Action Center source context (P0)	· Gantt expand/collapse + drill-down · Dashboard stacked charts · Notification dismiss + mark read	· Generate Report → PDF download · Documents page live content · Chat project creation UI (part 2)	· Chat project creation UI (part 3) · Empty states/responsiveness pass · UI polish + loading states
Joel	· Rewrite LLM extraction prompt (P0) · Wire approved actions → task creation (P0)	· Real Email OAuth connector (Gmail) · Email response monitoring loop	· Dependency completion notification · Improve risk scoring (owner signals) · Manager escalation hierarchy	· PDF report generation endpoint · Rate limiting on AI endpoints · Production calibration verification

CURRENT PRIORITIES

How to work together

Ship regressions are a shared responsibility
All three engineers will need a feature change. Agree on a single PR template that covers what changed, what could break, and how to test it. Joel's statement of the algorithm as a set of rules is the best example of this pattern. Adopt it.

Define API contracts before building UI
Fergus and Joel, when you add a new endpoint, write the response shape in a comment or a shared file before Anton builds the frontend against it. Anton currently assumes the SDF's array and the correlation id are already unwrapped. Agree the Content-Wrap data flow. Do it once, do it right.

Test against real transcripts, not toy examples
The LLM extraction quality is everything. Joel: build a test fixture from 3 real meeting transcripts (messy, incomplete, ambiguous) and run every extraction against it. Anton: if extraction gives you bad data, write proper empty states. This is the moat. It compounds with corrections over time.

Workers in most be split before Week 3
At 500 users, the worker PM will cause merge conflicts. Fergus: plan the split (extraction worker, execution worker, scheduler worker) in advance. Do it when it's boundaries. Wounds to (agent run, processing), escalations (scan), and handlers (job type dispatch). Takes 30 minutes and saves hours.

THE PERFECT TRUTH

Larry's moat is the feedback loop, not the features.

Every correction teaches the system. Confidence thresholds adjust per tenant. Pattern recognition improves with volume. Any time a PM watches Larry's decision, they know exactly how the next extraction should be. That compounding data advantage, accumulated in the solution layer, and it's the one thing competitors cannot copy once you have six months of real corrections in production.